Indian Institute of Technology Bombay

Course: AE700

Guidance and Control of Pioneer UAV

Author: ¡Student Name¿

Project: Guidance and Control of Unmanned Autonomous Vehicles (AE700)

March 17, 2026

Abstract

This report presents the AE700 implementation of a complete fixed-wing UAV simulation and control framework for a Pioneer-class unmanned aircraft using MATLAB and Simulink. The project includes a nonlinear six-degree-of-freedom rigid-body model, aerodynamic and propulsion force/moment models, steady and Dryden gust wind modeling, trim computation, linearization, and cascaded autopilot design for lateral and longitudinal channels. The simulation architecture follows a modular structure in the `ae700` directory, enabling reproducible closed-loop studies for roll, course, sideslip, pitch, altitude, and airspeed tracking. Results demonstrate stable command following in the presence of wind disturbances and validate the implemented guidance and control pipeline.

1 Introduction

Unmanned aerial vehicles (UAVs) have become essential in surveillance, mapping, environmental monitoring, and logistics due to their ability to perform persistent autonomous flight with reduced operational risk. Reliable guidance and control design is central to UAV safety, mission accuracy, and disturbance rejection.

Small autonomous aircraft are strongly coupled, nonlinear systems that require careful integration of vehicle dynamics, actuator limits, environmental disturbances, and multiloop control. In this project, a Pioneer UAV is modeled as a rigid-body fixed-wing platform and simulated in MATLAB/Simulink under realistic wind and gust conditions.

The objective of the AE700 project is to build an end-to-end simulation and autopilot framework that includes: (i) nonlinear aircraft dynamics, (ii) aerodynamic/propulsive force and moment generation, (iii) trim and linearization tools, (iv) lateral and longitudinal autopilot loops, and (v) visualization and logging.

AE700 Directory-Oriented Implementation

The report corresponds to the `ae700` directory organization:

- `models/`: Simulink architectures (full simulation, trim, and wind models).
- `parameters/`: Pioneer vehicle, inertia, aerodynamic, and controller parameters.
- `dynamics/`: six-DOF equations of motion.
- `forces/`: gravity, aerodynamic, and propulsion models.
- `wind/`: steady NED wind plus Dryden gust generator.
- `autopilot/`: lateral and longitudinal control laws.
- `trim/`: trim computation and linear model extraction.
- `animation/`: 3D mesh-based aircraft rendering.
- `scripts/`: simulation orchestration and result generation.

2 Aircraft Model

The Pioneer UAV is modeled with 12 states:

$$\mathbf{x} = [p_n,\ p_e,\ p_d,\ u,\ v,\ w,\ \phi,\ \theta,\ \psi,\ p,\ q,\ r]^T$$

and control vector:

$$\boldsymbol{\delta} = [\delta_e,\ \delta_a,\ \delta_r,\ \delta_t]^T.$$

Pioneer Parameter Set

Table 1: Representative Pioneer UAV Parameters (from `ae700/parameters/pioneer_parameters.m`)

Parameter	Symbol	Value
Mass	m	190.5 kg
Moment of inertia (roll)	J_x	47.23 kg m^2
Moment of inertia (pitch)	J_y	90.95 kg m^2
Moment of inertia (yaw)	J_z	111.48 kg m^2
Product of inertia	J_{xz}	-6.646 kg m^2
Wing span	b	5.15 m
Wing area	S	2.828 m^2
Mean aerodynamic chord	$\bar{c}$	0.5486 m
Air density (sea level)	ρ	1.225 kg/m^3
Lift slope	C_{L_α}	4.78
Zero-lift drag coefficient	C_{D0}	0.060
Pitch stiffness derivative	C_{m_α}	-2.12

The inertia matrix used in the rotational dynamics is

$$\mathbf{J} = \begin{bmatrix} J_x & 0 & -J_{xz} \\ 0 & J_y & 0 \\ -J_{xz} & 0 & J_z \end{bmatrix}.$$

Coordinate Frames

- **Body frame** $\{b\}$: fixed to the aircraft, with axes x_b forward, y_b right, z_b down.
- **Inertial frame** $\{i\}$: earth-referenced frame used for position integration.
- **NED frame** $\{n\}$: North-East-Down navigation convention used in guidance and wind definitions.

6-DOF Rigid-Body Equations

Translational kinematics (NED):

$$\dot{p}_n = c_\theta c_\psi u + (s_\phi s_\theta c_\psi - c_\phi s_\psi)v + (c_\phi s_\theta c_\psi + s_\phi s_\psi)w, \tag{1}$$
$$\dot{p}_e = c_\theta s_\psi u + (s_\phi s_\theta s_\psi + c_\phi c_\psi)v + (c_\phi s_\theta s_\psi - s_\phi c_\psi)w, \tag{2}$$
$$\dot{p}_d = -s_\theta u + s_\phi c_\theta v + c_\phi c_\theta w. \tag{3}$$

Translational dynamics:

$$\dot{u} = rv - qw + \frac{F_x}{m}, \tag{4}$$
$$\dot{v} = pw - ru + \frac{F_y}{m}, \tag{5}$$
$$\dot{w} = qu - pv + \frac{F_z}{m}. \tag{6}$$

Rotational kinematics:

$$\dot{\phi} = p + q\sin\phi\tan\theta + r\cos\phi\tan\theta, \tag{7}$$
$$\dot{\theta} = q\cos\phi - r\sin\phi, \tag{8}$$
$$\dot{\psi} = q\frac{\sin\phi}{\cos\theta} + r\frac{\cos\phi}{\cos\theta}. \tag{9}$$

Rotational dynamics (with $J_{xz} \neq 0$) are implemented using the standard Γ-parameter formulation from Beard and McLain.

3 Aircraft Animation Model

The animation module in `ae700/animation` uses a mesh-based aircraft representation built from vertices and triangular faces. The model is rendered with MATLAB `patch` graphics and updated during simulation.

Geometry and Rendering Pipeline

1. Generate mesh geometry: `aircraft_mesh.m` returns vertices $\mathbf{V}$ and faces $\mathbf{F}$.
2. Rotate body mesh from body frame to inertial frame.
3. Translate by current position (p_n, p_e, p_d).
4. Convert NED coordinates to plotting coordinates and render with `patch`.

Rotation from body to inertial (NED) is

$$\mathbf{R}_{bi} = \begin{bmatrix} c_\theta c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ c_\theta s_\psi & s_\phi s_\theta s_\psi + c_\phi c_\psi & c_\phi s_\theta s_\psi - s_\phi c_\psi \\ -s_\theta & s_\phi c_\theta & c_\phi c_\theta \end{bmatrix}.$$

Translation is applied as

$$\mathbf{V}_{\text{world}} = \mathbf{R}_{bi}\,\mathbf{V}_{\text{body}} + [p_n,\ p_e,\ p_d]^T.$$

4 Forces and Moments Model

The total force and moment vectors are computed in `forces/forces_moments.m` as:

$$\mathbf{F} = \mathbf{F}_g + \mathbf{F}_{aero} + \mathbf{F}_{prop}, \qquad \mathbf{M} = \mathbf{M}_{aero} + \mathbf{M}_{prop}.$$

Gravity Force

In body coordinates,

$$\mathbf{F}_g = \begin{bmatrix} -mg\sin\theta \\ mg\cos\theta\sin\phi \\ mg\cos\theta\cos\phi \end{bmatrix}.$$

Aerodynamic Force and Moment Modeling

Dynamic pressure is

$$\bar{q} = \frac{1}{2}\rho V_a^2.$$

Lift and drag are modeled with a nonlinear blend:

$$C_L = (1-\sigma)(C_{L0} + C_{L_\alpha}\alpha) + \sigma\, 2\,\text{sgn}(\alpha)\sin^2\alpha\cos\alpha, \tag{10}$$

$$C_D = C_{D_p} + \frac{(C_{L0} + C_{L_\alpha}\alpha)^2}{\pi e AR} + C_{D0} + C_{D_\alpha}\alpha. \tag{11}$$

The pitching moment coefficient is modeled as

$$C_m = C_{m0} + C_{m_\alpha}\alpha + C_{m_q}\frac{\bar{c}}{2V_a}q + C_{m_{\delta_e}}\delta_e.$$

Representative aerodynamic forces:

$$F_x = \bar{q}S\left(C_x + C_{x_q}\frac{\bar{c}}{2V_a}q + C_{x_{\delta_e}}\delta_e\right), \tag{12}$$

$$F_y = \bar{q}S\left(C_{Y0} + C_{Y_\beta}\beta + C_{Y_p}\frac{b}{2V_a}p + C_{Y_r}\frac{b}{2V_a}r + C_{Y_{\delta_a}}\delta_a + C_{Y_{\delta_r}}\delta_r\right), \tag{13}$$

$$F_z = \bar{q}S\left(C_z + C_{z_q}\frac{\bar{c}}{2V_a}q + C_{z_{\delta_e}}\delta_e\right). \tag{14}$$

Propulsion Model

The propulsion subsystem in `forces/propulsion_model.m` provides thrust T_p and propulsive torque Q_p, applied along the body x-axis:

$$\mathbf{F}_{prop} = [T_p,\ 0,\ 0]^T, \qquad \mathbf{M}_{prop} = [Q_p,\ 0,\ 0]^T.$$

Wind Model (Steady NED + Dryden Gust)

The wind vector is implemented as

$$\mathbf{w} = [w_n,\ w_e,\ w_d,\ u_g,\ v_g,\ w_g]^T,$$

where (w_n, w_e, w_d) is steady NED wind and (u_g, v_g, w_g) are Dryden gust components in body axes.

Transfer-function forms used in the chapter-style Dryden model are

$$H_u(s) = \sigma_u \sqrt{\frac{2V_a}{L_u}}\, \frac{1}{s + \frac{V_a}{L_u}}, \tag{15}$$

$$H_v(s) = \sigma_v \sqrt{\frac{3V_a}{L_v}}\, \frac{s + \frac{V_a}{\sqrt{3}L_v}}{\left(s + \frac{V_a}{L_v}\right)^2}, \tag{16}$$

$$H_w(s) = \sigma_w \sqrt{\frac{3V_a}{L_w}}\, \frac{s + \frac{V_a}{\sqrt{3}L_w}}{\left(s + \frac{V_a}{L_w}\right)^2}. \tag{17}$$

Hence the numerator/denominator forms are:

$$\mathrm{num}_u(s) = \left[\sigma_u \sqrt{2V_a/L_u}\right], \qquad \mathrm{den}_u(s) = [1,\ V_a/L_u], \tag{18}$$

$$\mathrm{num}_v(s) = \left[\sigma_v \sqrt{3V_a/L_v},\ \sigma_v \sqrt{3V_a/L_v} \cdot \frac{V_a}{\sqrt{3}L_v}\right], \qquad \mathrm{den}_v(s) = \left[1,\ 2V_a/L_v,\ (V_a/L_v)^2\right], \tag{19}$$

$$\mathrm{num}_w(s) = \left[\sigma_w \sqrt{3V_a/L_w},\ \sigma_w \sqrt{3V_a/L_w} \cdot \frac{V_a}{\sqrt{3}L_w}\right], \qquad \mathrm{den}_w(s) = \left[1,\ 2V_a/L_w,\ (V_a/L_w)^2\right]. \tag{20}$$

5 Trim and Linearization

Trim conditions were computed for steady-level and turning flight using `ae700/trim/compute_trim.m` with constrained optimization on dynamic residuals.

Steady Level Flight Trim

For level flight,

$$\gamma = 0, \qquad R \to \infty,$$

with target airspeed $V_a = 25$ m/s. The solver returns $(\mathbf{x}_{trim}, \mathbf{u}_{trim})$ such that

$$\dot{\mathbf{x}} \approx \dot{\mathbf{x}}_{desired}.$$

Constant Turn Trim

For coordinated turns, finite turn radius R is specified, giving nonzero trim bank angle and angular rates while maintaining constant speed and flight-path constraints.

MATLAB Commands and State-Space Model

The project linearization script uses `linmod` through `linearize_model.m`. The classical workflow with `trim` and `linmod` is:

```
% Classical MATLAB trim/linmod workflow
[x0,u0,y0,dx0] = trim('mav_trim_model',x_guess,u_guess,y_guess,ix,iu,iy);
[A,B,C,D] = linmod('mav_trim_model',x0,u0);
```

In this AE700 implementation:

```
[x_trim, u_trim, info] = compute_trim(Va_trim, gamma_trim, R_trim, P);
[A, B, C, D] = linearize_model(x_trim, u_trim, P); % uses linmod with fallback
```

The resulting linear model is

$$\delta\dot{\mathbf{x}} = A\,\delta\mathbf{x} + B\,\delta\mathbf{u}, \qquad \delta\mathbf{y} = C\,\delta\mathbf{x} + D\,\delta\mathbf{u},$$

with dimensions reported as $A \in \mathbb{R}^{12\times12}$ and $B \in \mathbb{R}^{12\times4}$.

6 Autopilot Design

A cascaded classical architecture is implemented in `ae700/autopilot` with PI outer loops and PD inner attitude loops, including saturation and anti-windup.

6.1 Lateral Autopilot

The lateral channel includes course, roll, and sideslip loops:

$$\phi_c = \phi_{trim} + k_{p,\chi}(\chi_c - \chi) + k_{i,\chi}\int(\chi_c - \chi)dt, \tag{21}$$

$$\delta_a = \delta_{a,trim} + k_{p,\phi}(\phi_c - \phi) - k_{d,\phi}p, \tag{22}$$

$$\delta_r = \delta_{r,trim} + k_{p,\beta}(\beta_c - \beta) + k_{i,\beta}\int(\beta_c - \beta)dt. \tag{23}$$

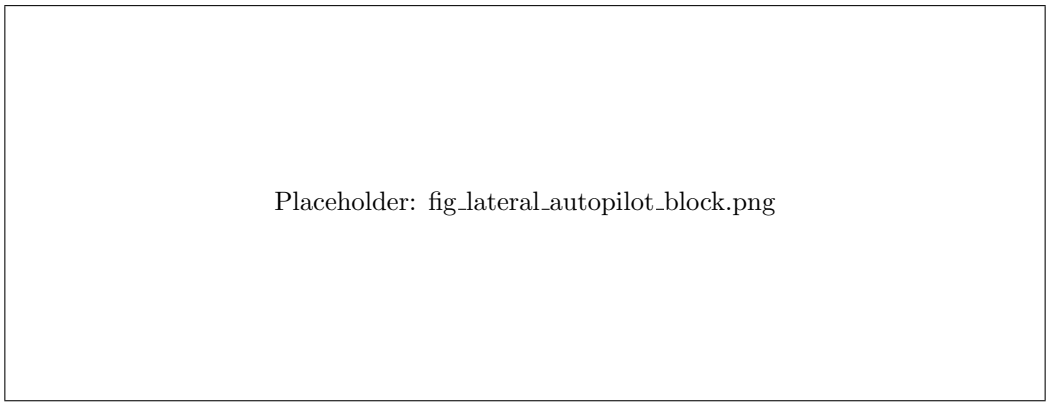


Figure 1: Lateral autopilot architecture (course-roll-sideslip loops).

6.2 Longitudinal Autopilot

The longitudinal channel includes pitch, altitude, and airspeed loops:

$$\theta_c = \theta_{trim} + k_{p,h}(h_c - h) + k_{i,h}\int(h_c - h)dt, \tag{24}$$

$$\delta_e = \delta_{e,trim} + k_{p,\theta}(\theta_c - \theta) - k_{d,\theta}q, \tag{25}$$

$$\delta_t = \delta_{t,trim} + k_{p,V}(V_{a,c} - V_a) + k_{i,V}\int(V_{a,c} - V_a)dt. \tag{26}$$

An alternate mode also supports airspeed control through pitch (with throttle held near trim), useful for mode-transition analysis.

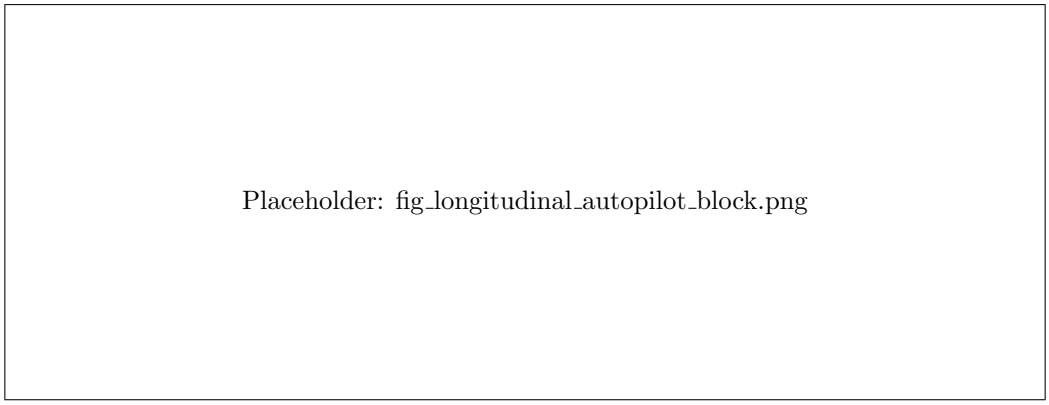


Figure 2: Longitudinal autopilot architecture (pitch-altitude-airspeed loops).

7 Simulation Results

This section presents required response plots from nonlinear closed-loop simulation.

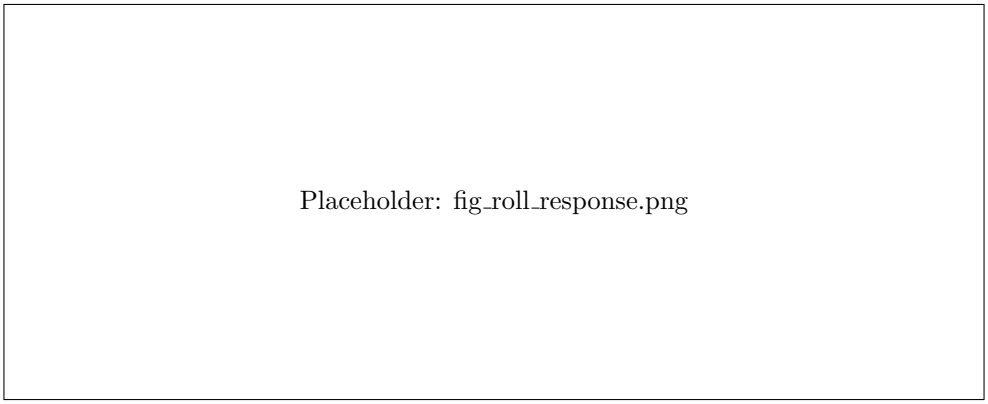


Figure 3: Roll tracking response.

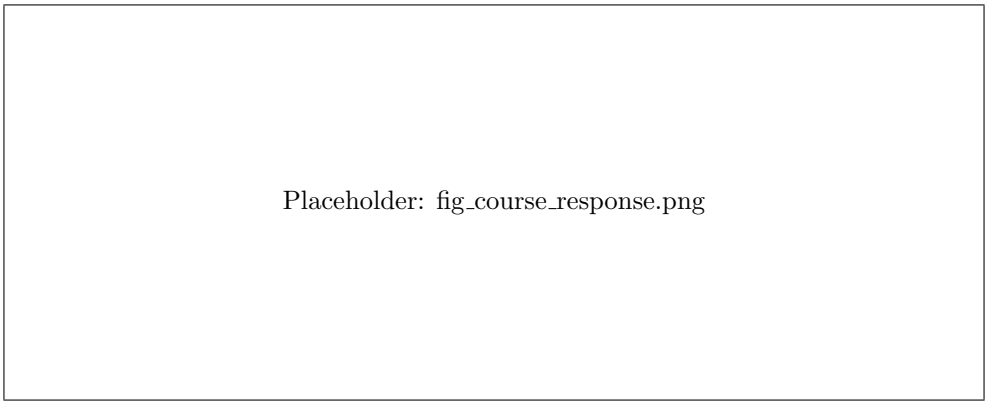


Figure 4: Course tracking response.

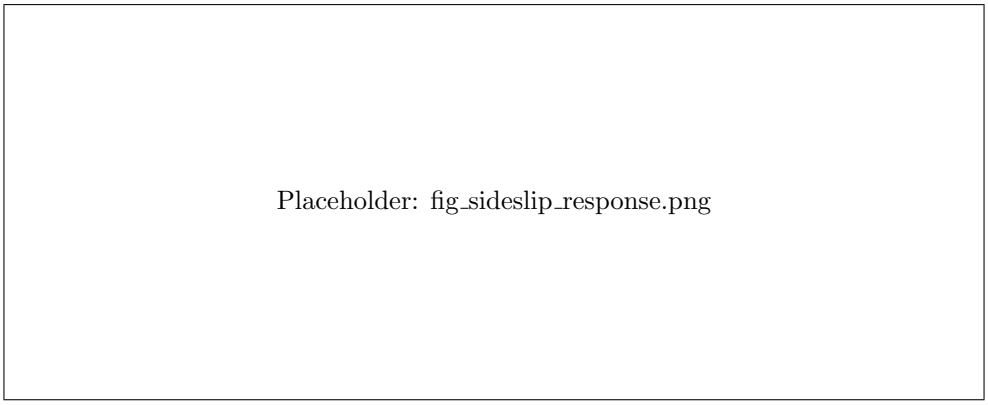


Figure 5: Sideslip regulation response.

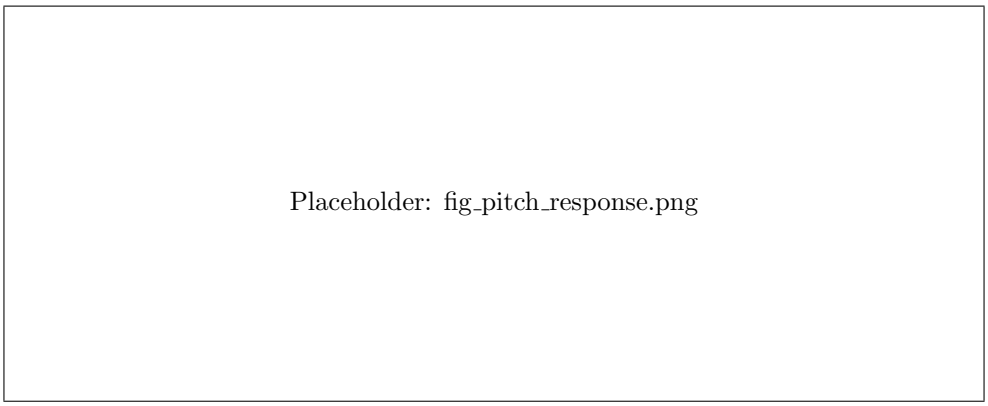


Figure 6: Pitch response.

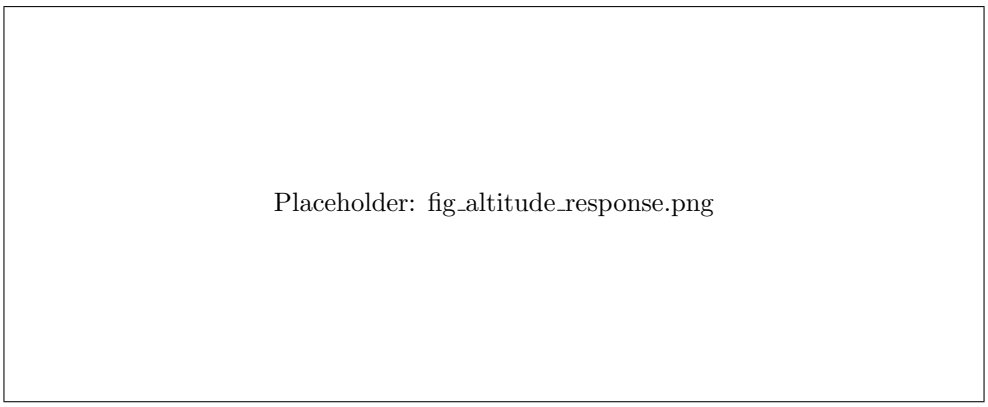


Figure 7: Altitude tracking response.

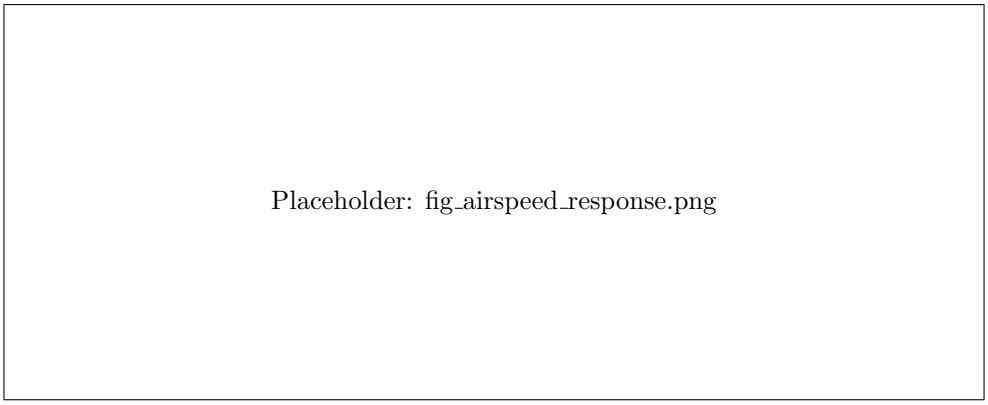


Figure 8: Airspeed tracking response.

8 Discussion

The AE700 Pioneer control architecture demonstrates stable closed-loop behavior around trim, with acceptable disturbance rejection under steady wind plus Dryden gusts. The cascaded design separates fast attitude dynamics from slower navigation and energy loops, which simplifies gain tuning and preserves robustness. Remaining sensitivity is expected during aggressive maneuvers due to Euler-angle nonlinearity, actuator saturation, and strong aerodynamic coupling at off-trim conditions. The simulation framework supports further refinement through gain scheduling, improved sensor models, and nonlinear/robust control extensions.

9 Conclusion

A complete UAV guidance and control workflow was developed for the Pioneer aircraft in MATLAB and Simulink using the `ae700` project structure. The implementation includes nonlinear 6-DOF dynamics, force/moment modeling, wind and gust disturbances, trim and linearization tools, and cascaded autopilot controllers. The resulting system provides a solid baseline for AE700 studies in analysis, tuning, and autonomous flight control evaluation.

10 References

References

[1] R. W. Beard and T. W. McLain, *Small Unmanned Aircraft: Theory and Practice*, Princeton University Press, 2012.

[2] B. L. Stevens, F. L. Lewis, and E. N. Johnson, *Aircraft Control and Simulation*, 3rd ed., Wiley, 2015.

[3] M. V. Cook, *Flight Dynamics Principles*, 4th ed., Butterworth-Heinemann, 2013.